

Chapitre 06 : Programmation Orientée Objet en Java v2.1

LPRGL3

PROJET INFORMATIQUE II



Objectifs

- Créer des classes, objets, attributs et méthodes
- Maîtriser les modificateurs d'accès (public, private)
- Utiliser l'héritage pour étendre les fonctionnalités
- Redéfinir les méthodes pour des besoins spécifiques
- Explorer le polymorphisme pour un traitement uniforme
- Découvrir les interfaces et leur rôle dans les contrats entre classes
- Développer des applications Java interactives et robustes

Prérequis

- Connaitre le langage Java
- Logiciels:
 - IntelliJ
 - Java JDK minimum 24
 - Un éditeur de texte

Sommaire

- Le monde des objets
- Interaction de l'utilisateur
- Héritage

Le monde des objets

Le monde des objets

Qu'est-ce que la POO ?

- **La POO (Programmation Orientée Objet) structure le code en "objets".**
- **Chaque objet représente une entité du monde réel (ex: personne, voiture).**
- **Objectif: refléter le problème dans l'organisation du code pour une meilleure compréhension et gestion.**
- **Un objet possède des attributs (données) et des méthodes (actions).**

Le monde des objets

Qu'est-ce que la POO ?

- Les objets sont comme des types de données améliorés et personnalisables.
- Avantages de la POO:
 - Code mieux organisé, lisible et maintenable.
 - Correspond à la nature orientée objet de Java.

Le monde des objets

Attributs

- **Les attributs représentent les données d'un objet (variables membres).**
- **Types d'attributs:**
 - Primitifs (entiers, caractères, etc.)
 - Objets (autres instances d'objets)
- **Exemple:**
 - Objet "livre" avec attributs "titre", "auteur", "nombreDePages"

Le monde des objets

Attributs

- Objet "bibliothèque" avec attribut "livres" contenant un tableau d'objets "livre"
- Définition des attributs:
 - Assigner des valeurs aux attributs avec l'opérateur "="
 - Exemple:
 - `book.nombreDePages = 234;` pour définir le nombre de pages du livre à 234

Le monde des objets

Attributs

- Accès aux attributs:
 - Opérateur "." pour accéder aux attributs d'un objet
 - Exemple:
 - `book.titre` pour accéder à l'attribut "titre" de l'objet "book"

Le monde des objets

Méthodes

- **Les méthodes sont des fonctions associées à des objets spécifiques.**
- Elles représentent les actions ou comportements d'un objet.
- Création de méthodes similaire à la création de fonctions.
- Appeler une méthode:
 - Utilisation du point "." pour appeler une méthode sur un objet.

Le monde des objets

Méthodes

- Syntaxe similaire à l'appel d'une fonction.
- Les méthodes peuvent recevoir des arguments (données d'entrée).
- Exemple:
 - Objet "book" avec une méthode "setBookmark(int pageNum)" pour définir un signet.
 - Appel de la méthode: `book.setBookmark(12);` pour définir un signet à la page 12.

Le monde des objets

Objets et classes

- Les objets encapsulent des données (attributs) et des comportements (méthodes).
- Les classes définissent la structure et les fonctionnalités des objets.
- **Une classe est un modèle pour créer des objets.**
- **Un objet est une instance d'une classe.**

Le monde des objets

Objets et classes

- Exemple:
 - Classe `Véhicule` avec attributs `couleur`, `puissance`, `nombreDeSièges`.
 - Objet `maVoiture` de type `Véhicule` avec valeurs spécifiques pour les attributs.
- Chaque classe dans un fichier .java.
- Créer une classe:
 - Mot-clé `class` suivi du nom de la classe.
 - Attributs (variables) de la classe.

Le monde des objets

Objets et classes

- Méthodes de la classe (fonctions).
- Créer un objet:
 - Mot-clé **new** suivi du nom de la classe et des parenthèses.
- Exemple:

```
class Vehicule {  
    String couleur;  
    int puissance;  
    int nombreDeSieges;
```

Le monde des objets

Objets et classes

```
public void demarrer() {  
    // Code pour démarrer le véhicule  
}  
public void accelerer() {  
    // Code pour accélérer le véhicule  
}  
}  
Vehicule maVoiture = new Vehicule();  
maVoiture.couleur = "bleue";  
maVoiture.puissance = 160;  
maVoiture.nombreDeSieges = 2;
```


Le monde des objets

Objets et classes

- Accéder aux attributs et méthodes d'un objet:
 - Opérateur `.` (point)
- Exemple:

```
maVoiture.demarrer();  
System.out.println(maVoiture.couleur);
```
- **Les chaînes de caractères sont des objets en Java (classe `String`).**
- Méthodes utiles pour les chaînes:
 - `length()` : longueur de la chaîne.
 - `equals(String s)` : comparaison d'égalité avec une autre chaîne.

Le monde des objets

Objets et classes

- Java est un langage orienté objet (OOP).
- **Les types primitifs sont également encapsulés dans des classes (`Integer`, `Double`, `Boolean`, etc.).**
- **Ces classes offrent des méthodes pour manipuler les types primitifs.**

Le monde des objets

La méthode main

- La méthode **`main`** est le point d'entrée d'un programme Java.
- Elle est généralement publique, statique et ne retourne rien.
- Syntaxe:

```
public static void main(String[] args) {
```

```
// Code à exécuter au démarrage du programme
```

```
}
```

Le monde des objets

La méthode main

- La méthode main permet de créer des objets et d'appeler des méthodes d'autres classes.
- **C'est la méthode obligatoire pour l'exécution d'un programme Java.**
- **public:** Permet d'appeler la méthode depuis n'importe où dans le programme.
- **static:** Ne nécessite pas d'objet pour s'exécuter.

Le monde des objets

La méthode main

- **void**: Ne retourne aucune valeur.
- **main**: Nom de la méthode.
- **String[] args**: Paramètre d'entrée (tableau de chaînes de caractères).

Le monde des objets

Créer des classes

- Les classes Java sont des fichiers .java.
- Un projet Java est un dossier contenant des fichiers .java.
- Les outils de développement Java (IDE) comme IntelliJ facilitent la création de classes.
- Créer un projet Java:
 1. Créer un nouveau projet Java nommé "Chapitre06 Classes" dans le dossier chapitre06classes. Ne pas ajouter de code exemple mais activer créer un dépôt Git. Sélectionnez IntelliJ comme outil de production.

Le monde des objets

Créer des classes

2. Sélectionner la version Oracle Open JDK minimum **24**.
Installer ce JDK si il est absent.
- Créer une classe
 1. Clic droit sur le dossier "src".
 2. Sélectionner "New" puis "Java Class".
 3. Nommer la classe Main.
 - Résultat:
 - Un fichier de classe `.java`` est créé dans le dossier "src".
 - La classe peut contenir des attributs et des méthodes.

Le monde des objets

Créer une méthode

- Une méthode est un bloc de code réutilisable.
- Elle regroupe des instructions pour effectuer une tâche spécifique.
- La syntaxe de base d'une méthode:

```
public static void nomMethode(paramètres) {  
    // Instructions de la méthode  
}
```


Le monde des objets

Constructeurs

- Les constructeurs sont des méthodes spéciales pour créer et initialiser des objets.
- Ils portent le même nom que la classe et n'ont pas de type de retour.
- Ils peuvent prendre des paramètres pour personnaliser l'initialisation.
- Exemple: `Game()` est le constructeur de la classe `Game`.

Le monde des objets

Constructeurs

- Un **constructeur par défaut** ne prend aucun paramètre.
- Il est optionnel, mais recommandé pour initialiser les attributs avec des valeurs par défaut.
- Exemple: `Game()` initialise l'attribut `score` à 0.
- Un **constructeur paramétré** prend des paramètres pour une initialisation personnalisée.

Le monde des objets

Constructeurs

- Permet de créer des objets avec des valeurs spécifiques dès le départ.
- Exemple: `Game(int startingScore)` initialise score avec la valeur donnée.
- **Les constructeurs ne sont pas appelés avec le point ".".**
- **Ils sont utilisés lors de la création d'objets avec le mot-clé new.**

Le monde des objets

Constructeurs

- Exemple: `Game tetris = new Game();` utilise le constructeur par défaut.
- Bonne pratique d'avoir un constructeur par défaut pour les cas typiques.
- Les constructeurs paramétrés offrent une flexibilité pour les cas moins courants.
- **Le constructeur par défaut devient obligatoire dès qu'un constructeur personnalisé est défini.**

Le monde des objets

Auto-référence

- **this** est un mot-clé en Java qui fait référence à l'objet courant.
- Permet d'accéder aux attributs et méthodes de l'objet à l'intérieur de ses propres méthodes ou constructeurs.
- Utile pour lever l'ambiguïté entre des noms d'attributs et de paramètres identiques.
- Exemple: `this.row` fait référence à l'attribut `row` de l'objet.

Le monde des objets

Auto-référence

- Lorsque le nom d'un paramètre est identique au nom d'un attribut, this permet de distinguer les deux.
- Sans this, la valeur du paramètre serait assignée à lui-même, sans modifier l'attribut.
- Exemple: `this.row = row`; assigne la valeur du paramètre `row` à l'attribut `row` de l'objet.

Le monde des objets

Auto-référence

```
class Position {  
    int row = 0;  
    int column = 0;  
    //constructeur  
    Position(int row, int column) {  
        this.row = row;  
        this.column = column;  
    }  
}
```

- La classe Position a des attributs row et column et un constructeur avec des paramètres du même nom.

Le monde des objets

Auto-référence

- Le constructeur utilise `this.row` et `this.column` pour assigner les valeurs aux attributs, évitant l'ambiguïté.
- Cette pratique rend le code plus clair et plus facile à comprendre.

```
// Constructeur :
```

```
ContactsManager(){
```

```
    this.friendsCount = 0;
```

```
    this.myFriends = new Contact[500];
```

```
}
```


Le monde des objets

Exemple : Gestionnaire de Contacts

- **Question 1**
- Pourquoi est-ce une bonne idée d'utiliser une variable String pour stocker un numéro de téléphone plutôt que d'utiliser un entier :
- (Il peut y avoir plus d'une bonne réponse)
 - A. Parce que vous ne pouvez pas comparer des entiers
 - B. Parce que les numéros de téléphone commencent par des 0 et que les entiers ne peuvent pas stocker les 0 de début
 - C. Parce que le plus grand entier est plus petit qu'un numéro de téléphone typique
 - D. Parce qu'une chaîne de caractères (String) nécessite moins de mémoire qu'un entier

Le monde des objets

Exemple : Gestionnaire de Contacts

- La classe ContactsManager gère l'ajout et la recherche de contacts.
- Elle utilise un tableau myFriends pour stocker les objets Contact.
- Un attribut friendsCount indique le nombre d'amis ajoutés au tableau.

```
class ContactsManager {  
    // Attributs :  
    Contact [] myFriends;  
    int friendsCount;  
}
```

Le monde des objets

Exemple : Gestionnaire de Contacts

- Un constructeur par défaut initialise friendsCount à 0 et crée un tableau myFriends de 500 éléments.
- Le compteur friendsCount s'incrémente à chaque ajout d'un nouveau contact.

```
// Constructeur :
```

```
ContactsManager(){
```

```
    this.friendsCount = 0;
```

```
    this.myFriends = new Contact[500];
```

```
}
```

Le monde des objets

Ajout de méthodes

- Classe ContactsManager :
 - Méthode addContact : Ajoute un nouveau contact au tableau.

```
void addContact(Contact contact){  
    myFriends[friendsCount] = contact;  
    friendsCount++; }  
}
```

- Méthode searchContact : Recherche un contact par nom et retourne l'objet Contact correspondant.

```
Contact searchContact(String searchName){  
    for(int i=0; i<friendsCount; i++){  
        if(myFriends[i].name.equals(searchName)){  
            return myFriends[i];  
        }  
    }  
    return null;}  
}
```

Le monde des objets

Ajout de méthodes

- Classe Main :
 - Point de départ du programme avec la méthode main.
 - Crée une instance de ContactsManager pour gérer les contacts.

```
class Main {  
    public static void main(String [] args){  
        ContactsManager myContactsManager = new  
        ContactsManager();  
    }  
}
```

Le monde des objets

Exercice 1 Gestionnaire de Contacts

- Créez le projet ContactsManager.
- Dans la méthode main:
 - Créez un objet ContactsManager appelé myContactsManager en utilisant le constructeur par défaut.
 - Créez une nouvelle variable Contact
 - Définissez l'attribut "name" avec le nom d'un ami.
 - Définissez l'attribut "phoneNumber" avec son numéro de téléphone.

Le monde des objets

Exercice 1 Gestionnaire de Contacts

- Appelez la méthode `addContact` de `myContactsManager` pour ajouter ce contact.
- Répétez les étapes 2 et 3 pour 4 autres contacts.
- Recherchez un contact en utilisant la méthode `searchContact` et affichez son numéro de téléphone.
- Publiez le projet sur Github et partagez le lien.
- Utilisez le fichier `README.md` de Github pour **documenter** le travail.
- Assurez-vous que le projet Github peut être importé sans erreurs et s'exécute.

Le monde des objets

Accès aux modificateurs

- Protéger le code des erreurs involontaires.
- Utiliser les modificateurs d'accès pour contrôler la visibilité.
- Exemple: photos publiques vs. privées.
 - **public**: accessible depuis d'autres classes.
 - **private**: accessible uniquement depuis la classe définie.
- Définir la visibilité avec `private` ou `public` avant la déclaration.

Le monde des objets

Attributs(public vs privé)

- Définir la visibilité avec private ou public avant le type d'attribut.
- **Par défaut, les attributs devraient être privés.**
- Initialiser les attributs privés dans le constructeur.
- Exemple: classe Book avec attributs title, author et isBorrowed.

Le monde des objets

Attributs(public vs privé)

- **Utiliser des méthodes getter et setter pour accéder aux attributs privés.**
- Les getters permettent de lire la valeur de l'attribut.
- Les setters permettent de modifier la valeur de l'attribut.
- Exemple: méthodes isBookBorrowed() et borrowBook() pour l'attribut isBorrowed.

Le monde des objets

Méthodes (publiques vs privées)

- Méthodes privées:
 - Utilisées pour organiser le code.
 - Invisibles et inaccessibles depuis l'extérieur de la classe.
 - Exemple: méthodes d'assistance.
- Méthodes publiques:
 - Représentent les actions de la classe.
 - Constituent l'interface avec le reste du programme.
 - Exemple: getUsername, isSamePerson.

Le monde des objets

Méthodes (publiques vs privées)

- Attributs de la classe Person définis comme privés.
- Seuls les méthodes et constructeurs de la classe peuvent les modifier.
- La méthode getId() est privée pour protéger les informations sensibles.

Le monde des objets

Méthodes (publiques vs privées)

- Les classes sont généralement définies comme **publiques**.
- Les classes privées nécessitent des connaissances sur les classes imbriquées.
- Visibilité par défaut des éléments: **package-public**.
 - Accessible uniquement aux classes du même package (dossier).
- **Recommandation: utiliser explicitement les qualificatifs d'accès.**

Le monde des objets

Question

- **Question 2**
- Sélectionnez tous les scénarios où vous devriez définir la méthode comme étant privée :
 - A. Une méthode d'assistance dans la classe Forme qui calcule la distance entre 2 points.
 - B. La méthode d'attaque dans la classe Pokémon.
 - C. Une méthode qui ne devrait être utilisée que par une autre méthode de cette classe.
 - D. Une méthode qui ne sera utilisée que par 1 autre classe.

Interaction de l'utilisateur

Interaction de l'utilisateur

Introduction

- Une application utile, est aussi interactive et amusante que possible.
- Par exemple, pour une application de gestionnaire de contacts, il faut permettre à l'utilisateur d'entrer les noms, numéros de téléphone et autres informations de ses amis.
- Voyons comment faire cela en Java.

Interaction de l'utilisateur

Lecture des entrées utilisateur

- 4 méthodes pour lire les entrées utilisateur en Java:
 - Entrée au moment de l'exécution (runtime)
 - Fichiers
 - Arguments de ligne de commande
 - Interface graphique (GUI)
- Couverture des 3 premiers types dans cette leçon.

Interaction de l'utilisateur

Lecteur d'entrée (Input Scanner)

- Le moyen le plus courant de lire l'entrée de l'utilisateur.
- L'utilisateur tape un message et Java le lit.
- Utiliser la **classe Scanner** de la **bibliothèque java.util**.
- Exemple: lire une chaîne de caractères, un entier ou une ligne entière.

Interaction de l'utilisateur

Lecteur d'entrée (Input Scanner)

- Créer un objet Scanner pour lire l'entrée du système.
- Utiliser la méthode **nextLine()** pour lire une chaîne de caractères.
- Utiliser la méthode **nextInt()** pour lire un entier.
- Exemple: lire l'adresse et la note de l'utilisateur.

Interaction de l'utilisateur

Lecteur d'entrée (Input Scanner)

```
System.out.println("Entrez votre note : ");  
Scanner scanner = new Scanner(System.in);  
int grade = scanner.nextInt();  
if(grade > 90){  
    System.out.println("Wow ! Bien joué !");  
} else {  
    System.out.println("Pas mal, mais tu peux faire mieux la  
prochaine fois !");  
}
```

Interaction de l'utilisateur

Le jeu du nombre à deviner

- L'ordinateur génère un nombre aléatoire entre 1 et 100.
- L'utilisateur a 10 essais pour deviner le nombre.
- L'utilisateur reçoit des indices ("Nombre proposé plus grand", "Nombre proposé plus petit").
- Le jeu félicite l'utilisateur s'il gagne, sinon il perd.

Interaction de l'utilisateur

Créer le jeu

- **Exercice 2 (A rendre)**
- Démarrez un nouveau projet et construisez vous-même le jeu de devinettes.
- Suivez ces étapes pour compléter l'exercice :
 - Générez un nombre aléatoire entre 1 et 100.
 - Créez une boucle qui demande à l'utilisateur de deviner un nombre.
 - Lisez l'entrée de l'utilisateur et comparez-la au nombre aléatoire.

Interaction de l'utilisateur

Créer le jeu

- Indiquez à l'utilisateur si le nombre deviné est supérieur ou inférieur au nombre aléatoire.
- S'il a deviné le bon nombre, mettez fin à la boucle et dites-lui qu'il a gagné.
- S'il a utilisé les 10 essais, mettez fin à la boucle et dites-lui qu'il a perdu.
- Publiez le projet sur Github et partagez le lien.
- Utilisez le fichier README.md de Github pour **documenter** le travail.
- Commentez intelligemment le code.
- Assurez-vous que le projet Github peut être importé sans erreurs et s'exécute.

Interaction de l'utilisateur

Lecteur de fichiers

- Une autre façon d'accepter les entrées au moment de l'exécution (runtime) est de le faire via des fichiers.
- Ces fichiers peuvent être des fichiers texte simples que l'utilisateur crée avec un éditeur de texte très basique (par exemple, Notepad sur Windows ou TextEdit sur Mac).

Interaction de l'utilisateur

Lecteur de fichiers

- Lire des données à partir de fichiers texte simples.
- Exemple: charger une liste de dépenses et calculer des statistiques.
- Utiliser la classe Scanner avec un objet File.

```
File file = new File("expenses.txt");  
Scanner fileScanner = new Scanner(file);
```
- La méthode **hasNextLine()** vérifie si le fichier contient des lignes.

Interaction de l'utilisateur

Lecteur de fichiers

- La méthode `nextLine()` lit une ligne du fichier.
- Traiter chaque ligne lue dans la boucle.

```
while (input.hasNextLine()) {  
    String line = input.nextLine();  
    // Utilisez cette ligne pour effectuer des calculs, des traitements,  
    etc.  
}
```

Interaction de l'utilisateur

Exercice 3 Nombre de mots (A rendre)

- Démarrez un nouveau projet et construisez vous-même un programme de comptage de mots.
- Suivez ces étapes pour compléter l'exercice :
 - Utilisez le lecteur de fichiers (File Scanner) pour lire le contenu de n'importe quel fichier volumineux.
 - Créez une boucle qui lit chaque ligne dans une chaîne de caractères (String).

Interaction de l'utilisateur

Exercice 3 Nombre de mots (A rendre)

- Séparez la chaîne en utilisant le caractère espace comme délimiteur.
- Comptez le nombre de mots dans cette ligne en utilisant la longueur du tableau créé après la séparation.
- Additionnez tous les mots de chaque ligne lue dans le fichier.
- Affichez le nombre total de mots.
- Effectuez plusieurs tests.

Interaction de l'utilisateur

Exercice 3 Nombre de mots (A rendre)

- Publiez le projet sur Github et partagez le lien.
- Utilisez le fichier README.md de Github pour documenter le travail.
- Commentez intelligemment le code.
- Assurez-vous que le projet Github peut être importé sans erreurs et s'exécute.

Interaction de l'utilisateur

Arguments de ligne de commande

- Fournir des entrées à un programme avant son exécution.
- Utilisé dans la déclaration de la méthode main.
- Accessible dans le tableau args.

- Exemple:

```
java MonProgramme argument1 argument2
```

- Rappel de la déclaration de la méthode main :

```
public static void main(String args[]){  
}
```

Interaction de l'utilisateur

Utiliser le terminal

- Exemple:

```
java meteo Tingrela
```

- pour la météo à Tingrela.
- Le code récupère les arguments dans le tableau args.

```
public static void main(String [] args){  
    if(args.length==0) {  
        System.out.println("Veuillez spécifier une ville");  
    }  
    else {
```

Interaction de l'utilisateur

Utiliser le terminal

```
String location = args[0];  
int temperature = 60 + (int)(Math.random()*10);  
System.out.println("La météo à "+location+" est "+  
temperature);  
}}
```


Interaction de l'utilisateur

Les exceptions

- Erreurs d'exécution causées par des entrées invalides ou des accès inexistantes.
- Gérer les exceptions pour avoir un programme plus robuste.
- Formalisation des erreurs d'exécution courantes.
- Des objets d'exception sont créés et transmis aux méthodes appelantes.

Interaction de l'utilisateur

Les exceptions

- Gérer les exceptions pour informer l'utilisateur et proposer des solutions.
- Déclarer les exceptions avec **throws** dans la signature de la méthode.
- Utiliser **`try ... catch`** pour entourer le code susceptible de lancer une exception.
- **Gérer les exceptions pour éviter les plantages et offrir une meilleure expérience utilisateur.**

Interaction de l'utilisateur

Gestion des exceptions

- Gérer les exceptions dans le bloc catch.

```
try {  
    openFile("somefile.txt");  
} catch (FileNotFoundException exception) {  
    // Gérer la situation en informant l'utilisateur de ce qui s'est passé  
    System.out.println("Impossible de trouver ce fichier");  
}
```

- Afficher un message à l'utilisateur ou relancer l'exception.

Interaction de l'utilisateur

Gestion des exceptions

- Relancer l'exception signifie que l'appelant doit la gérer.

```
try {  
    openFile("somefile.txt");  
} catch(FileNotFoundException exception) {  
    // Fuir la responsabilité  
    throw exception;  
}
```

- Plusieurs instructions catch pour différents types d'exceptions.

Interaction de l'utilisateur

Gestion des exceptions

- Gérer chaque type d'exception dans son propre bloc `catch`.

```
try {  
    openFile("somefile.txt");  
    array[index]++;  
} catch(FileNotFoundException exception) {  
    // Gérer tous les problèmes potentiels de type "fichier non trouvé"  
    ici.  
} catch(IndexOutOfBoundsException exception) {  
    // Gérer tous les problèmes potentiels de type "indice hors limites"  
    ici.  
}
```

Interaction de l'utilisateur

Gestion des exceptions

- Utile lorsque différentes exceptions peuvent survenir.
- Utiliser **catch(Exception)** pour capturer toutes les exceptions.

```
try {  
    openFile("somefile.txt");  
    array[index]++;  
} catch(Exception exception) {  
    // Gérer toutes les exceptions possibles ici  
}
```

Interaction de l'utilisateur

Gestion des exceptions

- Gérer toutes les exceptions dans un seul bloc catch.
- Utile lorsque le type d'exception spécifique est inconnu.

Interaction de l'utilisateur

Questions sur les exceptions

- Ce code est conçu pour demander à l'utilisateur un numéro de mois et afficher le nom abrégé du mois correspondant.
- Lisez attentivement le code, puis essayez de répondre aux questions ci-dessous.

Interaction de l'utilisateur

Questions sur les exceptions

- **Question 3**
- Quelle sera la sortie si l'utilisateur entre 3 :
 - A. "Mar"
 - B. "Apr"
 - C. "Input mismatch"
 - D. Aucune des réponses ci-dessus

Interaction de l'utilisateur

Questions sur les exceptions

- **Question 4**
- Quelle sera la sortie si l'utilisateur entre 99 :
 - A. "Index is out of bounds"
 - B. "Input mismatch"
 - C. "Dec"
 - D. Le programme va planter

Interaction de l'utilisateur

Questions sur les exceptions

- **Question 5**
- Quelle sera la sortie si l'utilisateur entre aaa :
 - A. "Dec"
 - B. "Index is out of bounds"
 - C. "Input mismatch"
 - D. Le programme plantera

Héritage

Héritage

Il y a plus dans la POO

- **L'héritage permet aux classes d'hériter des propriétés d'autres classes.**
- Favorise la réutilisation du code et réduit la redondance.
- Améliore l'organisation et la maintenabilité du code.
- Exemple de compte bancaire

Héritage

Il y a plus dans la POO

- Classe de base "CompteBancaire" avec attributs et méthodes communs.
- Classes dérivées "Compte Courant", "Compte Épargne", "Certificat de Dépôt".
- Chaque classe dérivée a ses propres attributs et méthodes uniques.
- L'héritage permet de partager le code commun et de réduire la duplication.

Héritage

Il y a plus dans la POO

- **La classe qui hérite est appelée classe enfant.**
- La classe héritée est appelée classe parente.
- La classe enfant hérite de tous les attributs et méthodes de la classe parente.

Héritage

Exercice 4 Le gestionnaire de banque(A rendre)

- Concevoir l'application BankManager
- Suivez ces étapes et vérifiez-les une fois que vous avez terminé, pour compléter cet exercice:
 - Créer une nouvelle classe nommée BankAccount (compte bancaire) avec les attributs account (compte) et balance (solde).

Héritage

Exercice 4 Le gestionnaire de banque(A rendre)

- Créer une nouvelle classe nommée CheckingAccount (compte courant) qui hérite de BankAccount avec un attribut supplémentaire nommé limit (limite).
- Répéter la même opération pour SavingsAccount (compte d'épargne) et COD (certificat de dépôt).
- Dans la méthode main, créer une instance de chacune des 3 classes enfants.
- S'assurer que vous pouvez accéder aux attributs account et balance (les définir et les lire).

Héritage

Exercice 4 Le gestionnaire de banque(A rendre)

- Publiez le projet sur Github et partagez le lien.
- Utilisez le fichier README.md de Github pour documenter le travail.
- Commentez intelligemment le code.
- Assurez-vous que le projet Github peut être importé sans erreurs et s'exécute.

Héritage

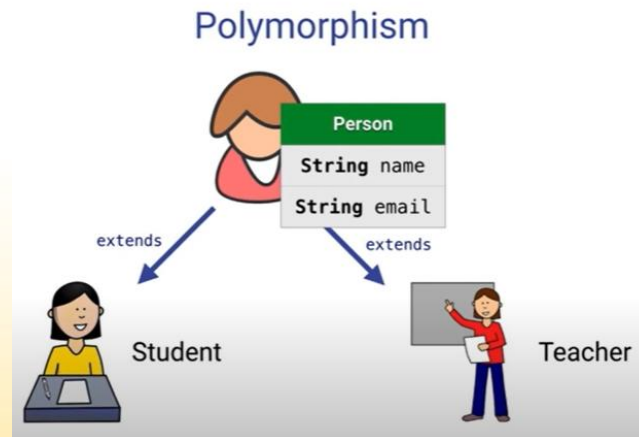
Le polymorphisme

- **Le polymorphisme permet aux objets de prendre plusieurs formes.**
- **Les objets hérités peuvent se comporter comme leur classe parente.**
- Flexibilité pour traiter les objets de manière polyvalente.
- Exemple de polymorphisme
- Classe "Personne" avec attributs "nom" et "email".

Héritage

Le polymorphisme

- Classes "Étudiant" et "Enseignant" héritant de "Personne".
- Les objets "Étudiant" et "Enseignant" sont aussi de type "Personne".



Héritage

Le polymorphisme

- Traiter les objets enfants comme des objets parents.
- Exemple: sac d'objets polymorphes
 - Sac avec limite de poids de 20 kg.
 - Différents objets ("Arbalète", "Clé", etc.) héritent de la classe "Objet".
 - Tous les objets ont un attribut "poids".
 - La méthode "peutAjouterObjet" du sac prend un paramètre de type "Objet".

Héritage

Le polymorphisme

- Implémenter la méthode "peutAjouterObjet" une seule fois.
- Accéder à l'attribut "objet.poids" dans la méthode.
- Fonctionne pour tous les objets héritant de "Objet".
- Exemple avec la classe "Arbalète"
 - Déclarer une variable de type "Arbalète".

Héritage

Le polymorphisme

- Passer l'objet "Arbalète" à la méthode "peutAjouterObjet".
- L'attribut "poids" de l'objet "Arbalète" est utilisé.

Héritage

Polymorphisme

- **Question 6**
- On vous donne une classe Book qui possède les attributs title et numberOfPages, ainsi que 2 autres classes Novel(Roman) et TextBook(ManuelScolaire) qui héritent de la classe Book.
- Laquelle des affirmations suivantes est/sont correcte(s)? (Il peut y avoir plus d'une bonne réponse)
 - A. `Book someBook = new Book();`
 - B. `Book someBook = new Novel();`
 - C. `Novel someNovel = new Book();`
 - D. `Novel someBook = new TextBook();`

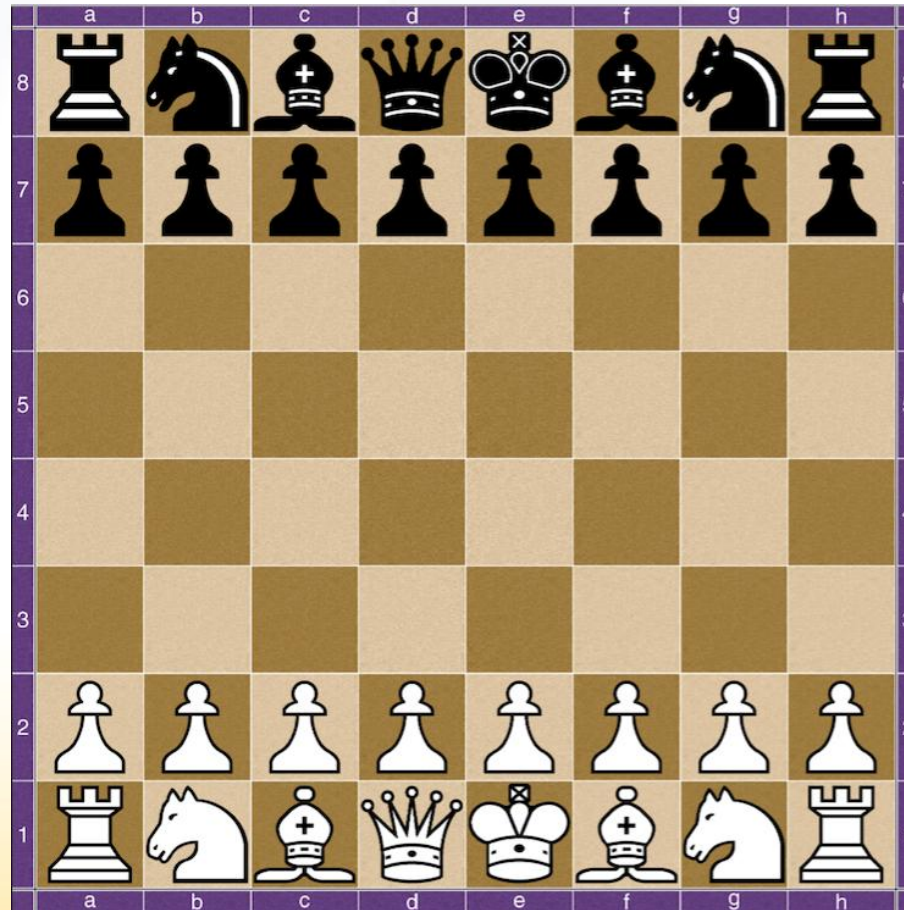
Héritage

Exemple du jeu d'échecs

- L'héritage permet d'étendre les fonctionnalités des classes.
- On peut aussi modifier le comportement des classes filles.
- Exemple du jeu d'échecs
- Classes pour les pièces d'échecs
- Classe de base "Pièce" pour toutes les pièces.
- Classes dérivées pour Roi, Reine, Tour, etc.

Héritage

Exemple du jeu d'échecs



Héritage

Exemple du jeu d'échecs

- Héritage pour partager des attributs et des méthodes.
- Classe "Jeu" pour représenter le plateau d'échecs.
- Classe "Position" pour stocker les coordonnées d'une case.
- Utilisation de "Position" dans la classe "Pièce".

```
class Piece {  
    Position position;  
}
```

Héritage

Exemple du jeu d'échecs

- Méthode pour vérifier si un déplacement est valide.
- Implémentation dans la classe "Pièce".

```
class Piece {  
    boolean isValidMove(Position newPosition){  
        if (newPosition.row>0 && newPosition.column>0  
            && newPosition.row<=8 && newPosition.column<=8) {  
            return true;  
        }  
    }  
}
```

Héritage

Exemple du jeu d'échecs

```
else {  
    return false;  
}  
}}
```

- Héritage pour que toutes les pièces l'utilisent.
- Exemple d'utilisation avec la Reine.

```
Queen queen = new Queen();  
Position testPosition = new Position(3,10);  
If (queen.isValidMove(testPosition)) {
```

Héritage

Exemple du jeu d'échecs

```
System.out.println("Yes, I can move there.");  
}  
Else {  
    System.out.println("Nope, can't do!");  
}
```

Héritage

Exemple du jeu d'échecs

- **Question 7**
- Que va afficher le code du slide précédent ?
 - A. "Yes, I can move there."
 - B. "Nope, can't do!"
 - C. Aucun des deux, ce code ne fonctionnera pas!

Héritage

Exemple du jeu d'échecs

- La méthode `isValidMove` est héritée de la classe `Pièce`.
- Chaque classe fille (Roi, Reine, Tour, etc.) la surcharge pour des mouvements spécifiques.
- Exemple: la Reine peut se déplacer en diagonale, horizontalement et verticalement.
- La surcharge permet une flexibilité et une spécialisation des mouvements des pièces.

Héritage

Redéfinition de méthodes

- **Permet de personnaliser le comportement d'une méthode héritée d'une classe parente.**
- La méthode redéfinie dans la classe enfant doit avoir la même signature (nom et paramètres) que celle de la classe parente.
- Le code à l'intérieur de la méthode redéfinie est modifié pour répondre aux besoins spécifiques de la classe enfant.

Héritage

Redéfinition de méthodes

- Chaque type de pièce d'échecs a des mouvements autorisés différents.
- La classe Piece (Pièce) définit une méthode isValidMove générique pour valider les mouvements.
- Les classes enfants, comme Rock (Tour) et Bishop (Fou), redéfinissent isValidMove pour implémenter leurs propres règles de mouvement.

Héritage

Redéfinition de méthodes

- Exemple de code classe Rock
- Exemple de code classe Bishop

Héritage

Exercice 5 Remplacez la méthode isValidMove

- C'est votre tour, remplacez la méthode isValidMove dans la classe Reine.
- Suivez ces étapes pour compléter cet exercice:
 - Dans la classe Reine, remplacez la méthode isValidMove
 - Ajoutez du code pour vérifier la validité du déplacement spécifique de la reine

Héritage Super

- Redéfinition de méthodes avec "**super**"
- Permet d'utiliser et d'étendre l'implémentation de la classe parente.
- Syntaxe: `super.nom_méthode(arguments)`.
- Evite la duplication de code.
- Favorise une hiérarchie de classes flexible.

Héritage Super

- Exemple: Redéfinition de isValidMove pour la Tour
- super.isValidMove vérifie les limites du plateau.
- Ensuite, la méthode vérifie si le mouvement est vertical ou horizontal.
- Cette implémentation étend la vérification de la classe parente.

Héritage Super

- Utilisation de "super" dans les constructeurs
- Appeler **super()** pour exécuter le constructeur de la classe parente.
- Initialiser les attributs spécifiques de la classe enfant.
- Assurer une initialisation cohérente des objets.

Héritage

Super

- **super()** est implicitement appelé si aucune instruction n'est présente dans le constructeur de la classe enfant.
- L'appel explicite de **super()** est recommandé pour une meilleure lisibilité et clarté du code.
- Si la classe parente n'a pas de constructeur par défaut, une erreur de compilation se produit.

Héritage

Déplacement de pièce

- **Question 8**
- Maintenant, si vous deviez implémenter la méthode responsable du déplacement réel de la pièce (c'est-à-dire mettre à jour l'attribut de position dans la classe de la pièce en lui fournissant une nouvelle position en paramètre), comment concevriez-vous une telle implémentation ?

Héritage

Déplacement de pièce

- Voici les options proposées pour implémenter la méthode move :
 - A. Implémenter la méthode move dans la classe Piece et la remplacer dans les classes filles.
 - B. Implémenter la méthode move dans la classe Piece en la définissant comme **final**.
 - C. Implémenter uniquement la méthode dans chacune des classes filles et non dans la classe Piece.

Héritage

Héritage multiple

- Permet de réutiliser et d'étendre le code existant.
- Favorise la modularité et la réutilisabilité du code.
- Principe de base de la programmation orientée objet.
- **Java n'autorise pas l'héritage multiple de classes.**

Héritage

Héritage multiple

- Ambiguïté potentielle en cas de méthodes homonymes dans les classes parentes.
- Solution: **Interfaces** en Java
- **Définissent des méthodes abstraites (sans implémentation) que les classes implémentent.**
- Permettent un "héritage multiple de fonctionnalités" via l'implémentation de plusieurs interfaces.
- Favorisent un couplage plus souple entre les classes.

Héritage

Les interfaces

- Définition de méthodes abstraites sans implémentation.
- Utilisées pour l'héritage multiple de fonctionnalités.
- Favorisent la flexibilité et le découplage du code.
- Création d'interfaces.

Héritage

Les interfaces

```
interface Mobile {  
    void seDeplacer();  
}  
  
interface Habitable {  
    boolean peutAccueillir();  
}
```

- Le mot-clé **interface** est utilisé à la place de class.
- Les méthodes sont définies avec des signatures mais sans corps.

Héritage

Les interfaces

- Implémentation d'interfaces

```
class Caravane implements Mobile, Habitable {  
    @Override  
    public void seDeplacer() {  
        System.out.println("La caravane se déplace...");  
    }  
    @Override  
    public boolean peutAccueillir() {  
        return true;  
    }  
}
```

Héritage

Les interfaces

- **Chaque méthode d'interface doit être implémentée.**
- **Avantages des interfaces**
 - Permettent l'héritage multiple de fonctionnalités.
 - Favorisent la flexibilité dans la conception des classes.
 - Promeuvent le polymorphisme et le découplage.
 - Améliorent la lisibilité et la maintenabilité du code.

Héritage

Interface Comparable

- Permet de comparer des objets du même type.
- Méthode unique **compareTo(objet)** pour définir l'ordre de tri.
- Utilisée pour le tri, la recherche et d'autres opérations.
- Implémentation de Comparable

Héritage

Interface Comparable

- La méthode **compareTo** compare l'objet "courant" à l'objet "spécifié".
- Renvoie un entier négatif si "courant" est inférieur, 0 s'ils sont égaux, un entier positif si "courant" est supérieur.
- La logique de comparaison dépend des critères de tri souhaités.

Héritage

Interface Comparable

- Exemple de tri

```
List<Livre> livres = new ArrayList<>();  
livres.add(new Livre(450, "Le Seigneur des Anneaux", "J.R.R.  
Tolkien"));  
livres.add(new Livre(390, "Harry Potter", "J.K. Rowling"));  
livres.add(new Livre(570, "Le Guide du Routard Galactique", "Douglas  
Adams"));  
Collections.sort(livres);  
System.out.println("Livres triés par nombre de pages :");  
for (Livre livre : livres) {  
    System.out.println(livre.titre + " (" + livre.nombreDePages + "  
pages)");  
}
```

Héritage

Interface Comparable

- Collections.**sort** utilise compareTo pour trier la liste.
- Le résultat montre les livres triés par ordre croissant de nombre de pages.

Héritage

Méthodes final

- **Empêche le remplacement des méthodes dans les classes filles.**
- **Garantit un comportement cohérent des méthodes héritées.**
- **Utilisé pour protéger les méthodes critiques ou de base.**
- **Exemple de méthode finale**

Héritage

Méthodes final

```
class Piece {  
    private double largeur;  
    private double hauteur;  
    public Piece(double largeur, double hauteur) {  
        this.largeur = largeur;  
        this.hauteur = hauteur;  
    }  
    public final double getSurface() {  
        return largeur * hauteur;  
    }  
}
```

Héritage

Méthodes final

```
class Salon extends Piece {  
    public Salon(double largeur, double hauteur) {  
        super(largeur, hauteur);  
    }  
    // Impossible de remplacer getSurface() ici  
}
```

- La méthode `getSurface()` est définie comme final dans la classe `Piece`.
- La classe `Salon` ne peut pas remplacer `getSurface()`.
- L'accès et l'utilisation de `getSurface()` depuis la classe `Salon` sont possibles.

Héritage

Attributs final

- Définit une constante accessible depuis n'importe où.
- La valeur ne peut pas être modifiée après l'initialisation.
- Utilisé pour les valeurs fixes et universelles.
- Exemple d'attribut final

```
public class MathLib {  
    public final double PI = 3.14;  
}
```


Héritage static

- **Permet de déclarer des attributs et des méthodes indépendants des objets.**
- **Appartient à la classe plutôt qu'aux instances.**
- **La valeur est partagée par tous les objets de la classe.**
- **Accès direct depuis la classe sans instancier d'objet.**

Héritage static

- Attributs statiques
- Le code ici
- count est déclaré comme attribut statique.
- Il est incrémenté par chaque objet créé.
- La valeur est partagée et accessible par tous les objets.

Héritage static

- Méthodes statiques
- Le code [ici](#)
- pow est déclarée comme méthode statique.
- Elle peut être appelée sans instancier d'objet.
- Elle appartient à la classe et non aux instances.

Héritage static

- Accès aux membres statiques
- Accès aux attributs statiques:
 - Nom de la classe.attribut
 - Ex: `MathUtils.PI`
- Accès aux méthodes statiques:
 - Nom de la classe.méthode(arguments)
 - Ex: `MathUtils.pow(2, 3)`

Héritage static

- Utilisations courantes:
 - Constantes universelles (ex: ``Math.PI``, ``System.out``)
 - Compteurs de classe
 - Fonctions utilitaires indépendantes des objets
 - Configuration de la classe
 - Variables partagées entre tous les objets

Héritage static

- **Exercice 6 Tester les Champs static**
- Créez un nouveau projet et créez une nouvelle classe appelée **Personne** qui contient un compteur static et un autre compteur non-statique.
- Incrémentez les deux compteurs dans le constructeur :

Héritage static

```
public class Personne {  
    public static int nbInstances;  
    public int nbLocal;  
    public Personne(){  
        nbInstances++;  
        nbLocal++;  
    }  
}
```

- Ensuite, dans la fonction main, créez plusieurs instances de la classe `Personne` et vérifiez les valeurs de chaque compteur.

Héritage static

```
public static void main(String[] args) {  
    Personne personne1 = new Personne();  
    Personne personne2 = new Personne();  
    Personne personne3 = new Personne();  
    Personne personne4 = new Personne();  
    // Affichez les valeurs des deux compteurs  
    System.out.println("(" + personne4.nbLocal + "," +  
        Personne.nbInstances + ")");  
}
```


Héritage static

- **Question 9**
- Terminez l'exercice 6 et répondez à cette question.
- Quelle valeur le code ci-dessus affichera-t-il pour les valeurs de (nbLocal, nbInstances) ?
 - A. (4,4)
 - B. (1,4)
 - C. (4,1)
 - D. (1,1)

Questions de fin de chapitre

- A traiter sur la plateforme en ligne.

Questions de codage

- A traiter sur la plateforme en ligne.

Contrôle des acquis

- A traiter sur la plateforme en ligne.

Pour aller plus loin

- HackerRank